

Table des matières

Exercices 03 — Écrire dans la base : DML et DDL	1
Questions courtes	1
Questions pratiques	2
Diagnostics	3
Questions à développer	4
Mises en situation	5

Exercices 03 — Écrire dans la base : DML et DDL

Comment travailler ces exercices. Répondez à chaque énoncé seul, puis vérifiez. Les questions courtes vérifient que le vocabulaire est en place ; les pratiques vous font créer, remplir et modifier des tables, tantôt sur le transactionnel, tantôt sur l'entrepôt ; les diagnostics vous montrent du code qui a fait des dégâts ; les mises en situation vous demandent de choisir une approche et de la défendre. Comptez quatre heures et demie pour y répondre, le corrigé non compris.

Les repères de terrain. Un énoncé marqué [OLTP] se traite dans PostgreSQL sur Adventureworks, et il écrit dans le schéma `labo03`. Un énoncé marqué [OLAP] se traite dans DuckDB sur l'entrepôt `adventureworksdw.duckdb`, schéma `dbo`, et il écrit dans des tables préfixées `LAB003_`. Sans marque, la question vaut pour les deux, ou se passe sur le moteur que l'énoncé nomme. Faites d'abord le nettoyage des deux labos, pour partir de bacs à sable vides, et n'écrivez jamais dans les tables d'AdventureWorks.

Le corrigé est dans `Corrige_03.md`. Répondez à l'énoncé en entier avant de l'ouvrir : c'est en butant sur une question qu'on découvre ce qu'on avait lu sans le comprendre.

Questions courtes

1. Le bac à sable. Qu'est-ce qu'un bac à sable ? Quelle convention la formation suit-elle pour le délimiter sur chaque terrain, et quelle commande supprime, sur chacun, tout ce qu'un labo a créé ?
2. La copie. Que recopie `CREATE TABLE ... AS`, et que ne recopie-t-il pas ? Donnez une situation où cette perte est sans importance et une où elle est grave.
3. Le type à l'insertion. Que devient 12,345 inséré dans une colonne `NUMERIC(6,2)`, et pourquoi 9 999,995 y est-il refusé ? Que se passe-t-il quand on insère 40 000 dans une colonne `SMALLINT`, puis ABCD dans une colonne `VARCHAR(3)`, sur PostgreSQL, puis sur DuckDB ?
4. Les cinq contraintes. Nommez-les, et dites pour chacune ce qu'elle interdit.

5. Les contraintes et NULL. Quelles contraintes laissent passer NULL ? Expliquez pourquoi CHECK (`prix >= 0`) accepte un livre sans prix.
6. Déclarée ou appliquée. Quelles contraintes DuckDB applique-t-il ? Et Snowflake ? Que devient, sur chacun, un doublon inséré dans une table dont la clé primaire est déclarée ?
7. La valeur par défaut. Dans quel cas une valeur DEFAULT s'applique-t-elle ? Que se passe-t-il si l'INSERT écrit explicitement NULL dans une colonne NOT NULL DEFAULT 'Montréal' ?
8. La liste de colonnes. Pourquoi faut-il nommer les colonnes d'un INSERT ? Décrivez deux façons dont un INSERT sans liste peut mal tourner.
9. DELETE et TRUNCATE. Donnez trois différences entre DELETE FROM `t` et TRUNCATE `t`.
10. La source en double. Que font UPDATE ... FROM et MERGE quand la source contient deux lignes pour la même clé, sur PostgreSQL, puis sur DuckDB ? Lequel préférez-vous pour un chargement, et pourquoi ?
11. La vue et la table de travail. Qu'est-ce qu'une vue garde, et qu'est-ce qu'une table de travail garde ? Laquelle des deux peut montrer un résultat périmé, et laquelle peut montrer un résultat faux dès que la table source l'est ?

Questions pratiques

Faites-les dans l'ordre : plusieurs questions réutilisent la table créée par une précédente. Avant chaque UPDATE ou DELETE, comptez les lignes visées, et comparez le compte rendu à ce compte.

P1. [OLTP] La copie de travail. Dans le schéma `labo03`, créez la table `produit` à partir de `production.product`, avec les colonnes `productid`, `name`, `productnumber`, `color`, `size`, `listprice` et `productsubcategoryid`. Vérifiez par une requête qu'elle n'a aucune contrainte, puis donnez-lui une clé primaire sur `productid`.

P2. [OLAP] La clé posée sur des données sales. Créez `LAB003_PRODUIT` à partir de `DimProduct`, avec `ProductKey`, `ProductAlternateKey`, `ProductSubcategoryKey`, `EnglishProductName`, `Color`, `Size` et `ListPrice`. Déclarez une clé primaire sur `ProductAlternateKey`, lisez la réponse du moteur, puis écrivez la requête qui l'explique. Concluez, et dites quelle colonne aurait dû porter la clé.

P3. [OLTP] Changer un type. Dans `labo03.produit`, la colonne `size` est du texte. Convertissez-la en `INTEGER`, les tailles en lettres devenant NULL. Avant la conversion, comptez les valeurs qui deviendront NULL alors qu'elles ne l'étaient pas ; après, comptez les tailles numériques et les produits sans taille.

P4. [OLTP] Une table faite pour durer. Créez `labo03.taux_cad`, qui range un taux de change par jour : la date, clé primaire ; le taux moyen et le taux de fin de journée, deux nombres exacts à quatre décimales, obligatoires et strictement positifs. Remplissez-la avec les taux du dollar canadien (`tocurrencycode = 'CAD'`) de `sales.currencyrate` antérieurs au 1er janvier 2024. Tentez ensuite d'insérer une seconde fois le 30 mai 2022.

P5. [OLTP] (avancé) Le chargement rejouable. Écrivez un MERGE qui charge dans `labo03.taux_cad` tous les taux du dollar canadien de `sales.currencyrate` : il insère les journées absentes, et ne met à jour une journée présente que si l'un de ses deux taux a changé. Vérifiez d'abord que la source n'a qu'une ligne par jour. Lancez le MERGE deux fois et comparez les deux comptes rendus.

P6. [OLTP] La correction par upsert. Le taux de fin de journée du 30 mai 2025 a été mal saisi : il vaut 1,5761, et le taux moyen de 1,5753 est juste. Écrivez la correction comme un upsert, qui insérerait la journée si elle manquait, et faites afficher par la commande elle-même la ligne écrite.

P7. [OLAP] Compter, écrire, vérifier. Le service des prix veut remplacer les prix absents de LAB003_PRODUIT par 0. Faites-le proprement : comptez les lignes visées, écrivez la modification dans une transaction, vérifiez le résultat, puis validez. Dites ensuite, chiffres à l'appui, pourquoi vous auriez déconseillé cette modification.

P8. [OLAP] UPDATE ... FROM sans surprise. Ajoutez à LAB003_PRODUIT une colonne SousCategorie, et remplissez-la avec le nom anglais de la sous-catégorie, tiré de DimProductSubcategory. Prouvez d'abord que la mise à jour ne peut pas tomber dans le piège de la source en double, puis vérifiez le nombre de lignes modifiées.

P9. [OLTP] La vue mensuelle. Créez la vue labo03.taux_cad_mensuel, qui donne pour chaque mois le nombre de jours cotés et le taux moyen arrondi à quatre décimales, en nommant ses colonnes. Comptez ses lignes. Puis, dans une transaction que vous annulerez, supprimez de la table les taux de 2025, et observez ce que devient la vue.

P10. [OLTP] Faire le ménage. Tentez de supprimer la table labo03.taux_cad. Expliquez le refus, puis supprimez tout le schéma labo03 d'une seule commande, et dites combien d'objets elle emporte.

P11. [OLAP] (avancé) L'expérience de la transaction. Écrivez, sur des tables LAB003_, une séquence de commandes qui établit si un CREATE TABLE lancé au milieu d'une transaction valide ou non l'INSERT qui le précède. Dites ce que vous attendez de chaque étape, lancez la séquence, et nettoyez.

Diagnosics

On vous montre du code qui a fait des dégâts, ou qui en fera. Dites ce qui cloche, pourquoi, et donnez le correctif.

E1. [OLTP] La promotion de trop. Une analyste veut baisser de 10 % le prix des trois casques Sport-100 dans sa copie de travail. Elle lance, dans pgAdmin :

```
UPDATE labo03.produit SET listprice = listprice * 0.9;
```

Le compte rendu affiche UPDATE 504. Qu'est-il arrivé ? Que peut-elle faire maintenant, et quelles habitudes l'auraient protégée ?

E2. Le MERGE qui plante. Depuis un an, sur l'entrepôt Snowflake d'une entreprise, un traitement de nuit met à jour les courriels de la dimension client :

```
UPDATE DIM_CLIENT AS d
SET Courriel = s.Courriel
FROM STG_CLIENT AS s
WHERE s.CustomerKey = d.CustomerKey;
```

Un collègue le récrit en MERGE pour insérer aussi les nouveaux clients. Dès le premier soir, le traitement échoue sur une erreur de ligne en double dans la source. L'équipe propose de revenir à l'ancien UPDATE, « qui, lui, n'a jamais planté ». Diagnostiquez, dites ce que vous faites, et ce qui se serait passé si le même MERGE avait tourné sur DuckDB.

E3. Le chiffre d'affaires qui double. Sur un entrepôt Snowflake, après la relance d'un chargement tombé en panne, un rapport montre des ventes doublées pour une partie des clients. La dimension DIM_CLIENT a pourtant une clé primaire déclarée sur CustomerKey, et le responsable affirme que « la base aurait refusé les doublons ». Expliquez le mécanisme, et écrivez la requête qui le prouve.

E4. La sauvegarde qui a tout validé. Sur un entrepôt Snowflake, avant une purge, un analyste écrit :

```
BEGIN;  
DELETE FROM CLIENT WHERE DateFirstPurchase < '2014-01-15';  
CREATE OR REPLACE TABLE CLIENT_SAUVEGARDE AS SELECT * FROM CLIENT;  
SELECT COUNT(*) FROM CLIENT;  
ROLLBACK;
```

Il comptait regarder le compte, puis tout annuler. Après le ROLLBACK, les clients supprimés ne sont pas revenus, et la sauvegarde ne les contient pas non plus. Expliquez les deux erreurs, réécrivez la séquence, et dites laquelle des deux erreurs aurait subsisté sur DuckDB.

E5. [OLTP] Les conversions qui dérivent. Une table de taux de change a été créée ainsi, puis chargée avec les taux du dollar canadien de sales.currencyrate :

```
CREATE TABLE labo03.taux (  
    jour DATE PRIMARY KEY,  
    taux NUMERIC(6,2) NOT NULL  
);
```

Elle sert à convertir des factures en dollars canadiens, et les montants convertis s'écartent de ceux du service financier, de plusieurs centaines de dollars sur les grosses factures. Aucune erreur n'a jamais été levée. Diagnostiquez, puis corrigez sans perdre davantage d'information.

E6. [OLTP] L'historique inversé. Un script mensuel archive les promotions :

```
INSERT INTO labo03.promotion_historique  
SELECT specialofferid, description, discountpct, type, category, startdate, enddate  
FROM sales.specialoffer;
```

Il fonctionne depuis deux ans. Le mois dernier, un collègue a recréé la table d'archive « à l'identique », en plaçant la catégorie avant le type, dans un ordre qu'il trouvait plus logique. Depuis, le rapport par catégorie affiche des catégories nommées « Volume Discount » et « Seasonal Discount ». Expliquez, et corrigez le script.

Questions à développer

Un court paragraphe de raisonnement, pas un mot juste.

A. La contrainte qu'on n'applique pas. D'après sa documentation, Snowflake déclare des clés primaires, des contraintes UNIQUE et des clés étrangères sans les appliquer, alors que DuckDB, lui aussi un moteur d'entrepôt, les applique. Défendez le choix de Snowflake du point de vue d'un entrepôt, puis dites ce qu'il exige d'une équipe qui charge des données, et où ce contrôle doit vivre.

B. DELETE contre TRUNCATE. Comparez les deux commandes du point de vue du volume, des clés étrangères, des transactions et de ce qu'on veut garder. Donnez une situation où TRUNCATE est le bon choix et une où ce serait une faute.

C. Le déploiement d'une migration. Une migration en trois étapes ajoute une colonne, la remplit par un UPDATE, puis la déclare NOT NULL. La troisième étape échoue, parce que quelques lignes sont restées vides. Décrivez l'état de la base après l'échec sur PostgreSQL ou DuckDB, puis sur Snowflake, et ce que chaque moteur vous oblige à prévoir avant de lancer la migration.

Mises en situation

On décrit une entreprise et une question métier. À vous de choisir l'approche, de l'écrire, et surtout de la justifier. Il y a plusieurs bonnes réponses, mais il y a des réponses indéfendables.

M1. Coopérative laitière du Bas-Saint-Laurent, le chargement des livraisons. Chaque nuit, la coopérative reçoit un fichier des livraisons de lait des sept derniers jours, parce que les fermes corrigent parfois une livraison déjà transmise. La table des livraisons de l'entrepôt compte cinq ans d'historique. Deux développeurs s'opposent : l'un veut supprimer les sept derniers jours et les réinsérer, l'autre veut un MERGE sur le couple ferme et jour.

Tranchez. Traitez trois cas : une livraison que la ferme a retirée du fichier, un fichier livré deux fois la même nuit, et un fichier qui contient deux lignes pour la même ferme et le même jour.

M2. Agence de voyages de Québec, l'unicité que plus personne ne vérifie. L'agence a migré sa dimension client d'une base transactionnelle vers Snowflake. Sur l'ancienne, la clé primaire bloquait les doublons ; sur Snowflake, trois mois plus tard, 2 % des clients sont en double faux.

Dites où et comment vous contrôleriez désormais l'unicité, à quel moment du chargement, et ce que vous feriez d'un chargement qui échoue au contrôle. La formation Analytics Engineering fournit l'outil ; ici, on veut la stratégie.

M3. Centre de services scolaire des Hautes-Laurentides, le tableau de bord des absences. Le tableau de bord lit une requête qui joint les absences, les élèves et les horaires, et qui prend quatre minutes. Une conseillère propose d'en faire une vue, un analyste une table de travail recalculée chaque nuit. Les directions consultent le tableau chaque matin, et les absences se saisissent toute la journée.

Tranchez, en disant ce que chaque option montre à 10 h, ce qu'elle coûte, et ce qui se passe le jour où la requête contient une erreur.

M4. (avancé) Transporteur de Saint-Hyacinthe, la colonne à corriger en production. Dans la base PostgreSQL de répartition, le poids des envois est rangé en VARCHAR, avec des valeurs comme « 1250 », « 980 kg » et « inconnu ». Deux vues et un rapport de facturation lisent la colonne. On vous demande de la passer en NUMERIC(8, 2) sans interrompre la facturation.

Décrivez vos étapes dans l'ordre, les commandes que vous emploieriez, ce que vous vérifiez entre deux étapes, et ce qui vous permet de revenir en arrière.

M5. Quincaillerie en ligne de Sherbrooke, 14 h 05. Un employé vient de lancer UPDATE produit SET prix = 9.99; sur la base PostgreSQL du site, en oubliant le WHERE. Le site affiche tous les articles à 9,99 \$, et des commandes commencent à entrer.

Dites ce que vous faites dans les cinq premières minutes, puis dans l'heure. Dites ce qui changerait si la même erreur avait frappé la table des prix de l'entrepôt Snowflake, qui alimente les rapports. Et dites ce que vous changez ensuite pour que ça ne se reproduise pas.